The Impact of Anomaly Detection with Isolation Forest on Random Forest and CNN-LSTM Classifiers in Intrusion Detection Systems

Ana Marcillo-Vera[1][0009−0006−6999−7664],
Juan Pablo Astudillo-León[1][0000−0002−7291−3584],
Iván Reyes-Chacón[2][0009−0002−2731−5531],
Paulina Vizcaino-Imacaña[2][0000−0001−9575−3539], and
Manuel Eugenio Morocho-Cayamcela[1,2][0000−0002−4705−7923]

[1] Yachay Tech University, School of Mathematical and Computational Sciences, DeepARC Research, Hda. San José s/n y Proyecto Yachay, Urcuquí, Ecuador, 100119
{ana.marcillo, jastudillo, mmorocho}@yachaytech.edu.ec

[2] International University of Ecuador UIDE, Faculty of Digital Engineering and Emerging Technologies, Quito, Ecuador, 170411
{ireyes, pvizcaino, mamorochoca}@uide.edu.ec

Abstract. Intrusion detection systems (IDS) are crucial for network security, detecting real-time malicious activities. This study investigates the impact of anomaly detection using *isolation forest* on the performance of two classifiers: the simple, interpretable *random forest* and the more complex *CNN-LSTM neural network*. The methodology follows two distinct paths: one that applies extensive preprocessing steps including stratified subsampling, PCA, and SMOTE before using *isolation forest*, and another that applies *isolation forest* as the initial step. In both cases, the data is separated into normal and anomalous sets for subsequent classification. Our results show that *isolation forest* effectively filters-out anomalous patterns from benign traffic, improving the downstream classification performance without introducing false positives. *Random forest* achieved high accuracy, recall and an F1-score of 0.97 across both data conditions, demonstrating superior computational efficiency. While *random forest* is faster for lightweight deployment, *CNN-LSTM* provides competitive results, particularly in recognizing complex attack signatures. These findings highlight the trade-off between simplicity and expressive power in IDS classifiers. Future research should focus on deploying this hybrid pipeline in real-time environments and assessing its robustness under adversarial scenarios, as traditional performance metrics do not account for this critical vulnerability.

Keywords: Intrusion detection systems (IDS), random forest, isolation forest, CNN-LSTM, PCA, SMOTE.

1 Introduction and Literature Review

Intrusion Detection Systems (IDS) are essential for protecting modern networks against diverse and evolving cyber threats. Traditional signature-based methods, although effective for known attacks, are limited in detecting novel threats due to their reliance on predefined patterns [27]. In contrast, anomaly-based detection techniques offer a promising alternative by identifying deviations from normal behavior [16], though they still face challenges in real-time applications.

This study employs the *CIC-IDS2017 dataset*, a realistic and comprehensive benchmark that simulates five days of network activity from 25 users using protocols such as HTTP, FTP, SSH, and email [23]. The infrastructure includes a full network topology—routers, switches, firewall—and diverse operating systems (Windows, Ubuntu, macOS), enabling the capture of attack types such as Brute Force, DoS, DDoS, Infiltration, and Botnet. Our research proposes and compares two processing paths for anomaly detection and classification, integrating Isolation Forest as a key preprocessing step. Isolation Forest is an unsupervised anomaly detection method that isolates points in the feature space using randomly constructed trees, making outliers easier to detect [26]. Using a contamination rate of 0.10, we separate the dataset into normal and anomalous samples, which are then processed using PCA for dimensionality reduction and SMOTE for class balancing.

While recent deep learning models achieve high accuracy with this dataset [10], they often come with high computational costs and limited interpretability [9]. Simpler models like Random Forest have shown competitive results with greater efficiency and transparency [7]. This highlights the need to balance performance with real-world applicability. We evaluate two models: a Random Forest classifier and a CNN-LSTM architecture. The CNN-LSTM combines convolutional layers for spatial pattern extraction with LSTM units for capturing temporal dependencies in traffic sequences [11, 25]. Both models are trained on preprocessed datasets and evaluated using accuracy, F1-score, and ROC-AUC.

Researchers have focused on developing scalable and intelligent systems capable of detecting known and novel threats in network traffic. Table 1 summarizes relevant studies highlighting different intrusion detection approaches, focusing on neural networks, machine learning techniques, and hybrid systems.

In the paper [12], a hybrid IDS is proposed that combines anomaly detection using Spark ML and a misuse detection module based on a convolutional-LSTM (Conv-LSTM) network. The approach leverages global and local latent features to enhance detection accuracy, achieving a performance of 97.29% on the ISCX-UNB dataset. The authors demonstrate the system's scalability and precision, emphasizing the Conv-LSTM's strength in addressing temporal and spatial network traffic patterns. This two-stage IDS shows that combining machine learning for anomaly detection and deep learning for misuse detection can outperform traditional approaches. Relevant work is the MLP4NIDS paper [20], where the authors propose a multi-layer perceptron (MLP) neural network to detect intrusions in the CIC-IDS2017 dataset. The proposed model achieves an impressive accuracy of 99.95%, outperforming traditional machine learning methods. The

Table 1: Summary of Related Works

Ref.	Algorithm	Dataset	Acc. (%)	Prec. (%)	Rec. (%)	F1-Sc. (%)
[12]	Conv-LSTM + Spark ML	ISCX-UNB	97.29	97.50	97.29	97.25
[20]	MLP	CIC-IDS2017	99.95	99.99	99.35	99.38
[18]	LSTM + SMOTE	CIC-IDS2017	99.33	86.43	98.22	91.95
[13]	LSTMAE + Spark MLlib	ISCX-UNB	97.52	96.53	97.52	97.02
[28]	MSC-BiLSTM + PCA	CIC-IDS2017	98.71	95.97	–	90.22
[24]	AE + IForest + PCA	CIC-IDS2017	97.75	94.59	94.78	97.78
[5]	RF + RUS/SMOTE/ADASYN	CIC-IDS2017	–	95.45	92.03	94.56
[7]	RF + Feature Selection	UNSW-NB15	99.67	–	–	–

study underscores the capability of simple neural network architectures, such as MLP, to provide effective intrusion detection with a low false positive rate of 0.08%. The results show that MLP can be an efficient alternative to more complex models, especially when computational resources are limited.

Nevertheless, several works have highlighted the vulnerability of machine learning-based NIDS to adversarial attacks, where small input perturbations can lead to misclassification. However, [1, 2] argue that many of these studies rely on unrealistic threat models, such as assuming complete attacker knowledge or unlimited system access. Their research introduces a more grounded framework for modeling feasible adversarial scenarios, which is crucial for building robust and practical intrusion detection systems.

2 Methodology

Our research's primary objective is to assess the Impact of Anomaly Detection with Isolation Forest on Random Forest and CNN-LSTM Classifiers in Intrusion Detection Systems. The general methodology is illustrated in Figure 1, and it follows two distinct paths to process and classify network traffic data. The first path takes a more direct approach, applying Isolation Forest on the raw dataset to detect anomalies without prior scaling or PCA. Once anomalies are detected, PCA is applied to reduce dimensionality, and further sampling and SMOTE techniques are used to balance the dataset. Both paths are designed to assess the impact of anomaly detection on model performance, with the final comparison determining if the inclusion of Isolation Forest enhances classification results for Random Forest and CNN-LSTM. The second path focuses on preprocessing steps such as feature scaling, dimensionality reduction through Incremental PCA, and balancing the dataset using manual sampling and SMOTE.

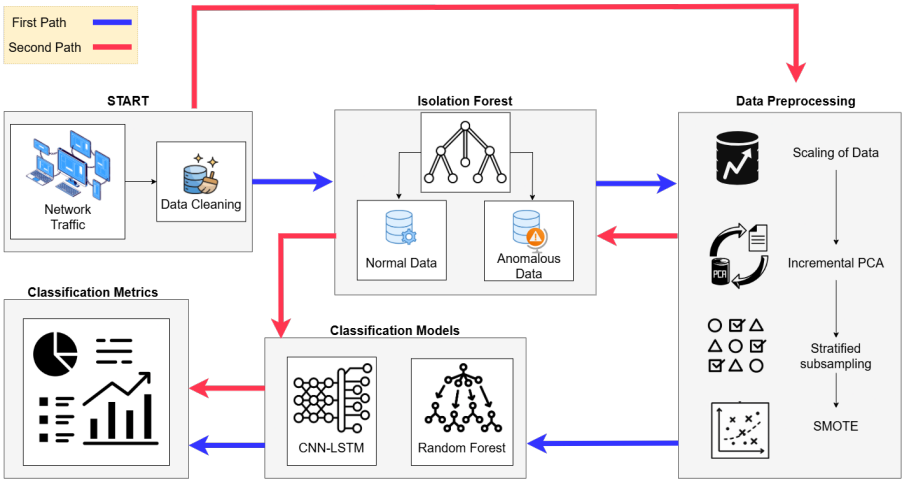


Fig. 1: Diagram illustrating the two paths in the intrusion detection framework. In the first path, data is separated using Isolation Forest before any preprocessing, while in the second path, preprocessing (scaling, PCA, and balancing) is applied before data separation. Both paths ultimately lead to classification models, Random Forest and CNN-LSTM, for generating performance metrics.

After these transformations, the dataset is split into normal and anomalous data using Isolation Forest, followed by model training with Random Forest and CNN-LSTM. This path aims to reduce dimensionality and improve the quality of features before feeding them into the models. The proposed methodology is organized into several stages, starting with data preprocessing, then anomaly detection using Isolation Forest, and finally classification using a Convolutional Neural Network with Long Short-Term Memory (CNN-LSTM)

The *CIC-IDS2017* dataset is a comprehensive intrusion detection dataset designed to address the limitations of previous datasets. It includes benign and modern attack traffic that closely resembles real-world network traffic [23].

It was designed with several key features in mind. First, it utilized the B-Profile system to model human interaction behavior, which allowed for the generation of naturalistic benign traffic. The data was captured over 5 days, from July 3–7, 2017. In terms of attack types, the dataset includes several categories such as Brute Force (FTP, SSH), DoS/DDoS, Heartbleed, Web Attacks (Brute Force, XSS, SQL Injection), Infiltration, and Botnet.

The CIC-IDS2017 dataset stands out as a benchmark for intrusion detection research due to its realistic traffic patterns, inclusion of contemporary attack types, and detailed labeling [21]. Its availability in both raw PCAP and preprocessed flow formats allows for flexible experimentation and reproducibility.

Figure 2 highlights this disparity, showing that benign traffic constitutes approximately 83.12% of the data, while attack traffic accounts for 16.88%. Fur-

thermore, when analyzing traffic across different days, it becomes apparent that benign traffic dominates every day, with no specific attack type overwhelmingly present. The closest to this are the DDoS and Port Scan attacks, which show higher volumes of traffic.

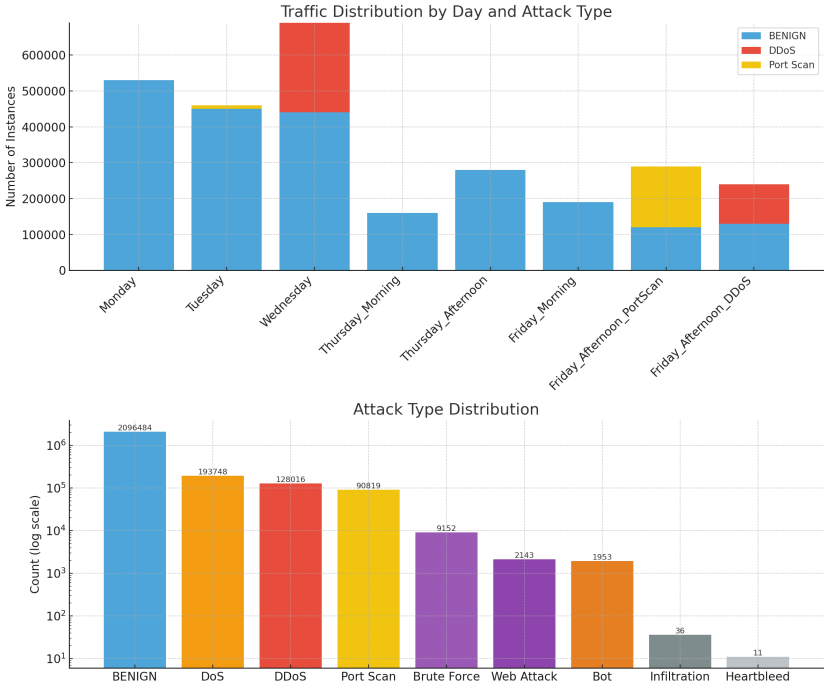


Fig. 2: Traffic distribution over days

The next step involved label mapping. Specific attack labels, such as DoS Hulk and DoS GoldenEye, were grouped into broader categories, such as DoS, to simplify analysis. Inconsistent naming conventions, particularly variations in Web Attack types, were standardized to ensure uniformity across the dataset. This consolidation helped reduce noise, balance the dataset, and ultimately improved the performance of machine learning models. Finally, there are several important considerations when working with the dataset. First, Heartbleed is a rare exploit that targets vulnerabilities in OpenSSL encryption, making it particularly challenging to detect due to its subtle behavior in network traffic [6].

2.1 First Path

The first path involves anomaly detection with the Isolation Forest algorithm, applied directly on the cleaned data to separate the dataset into anomalous

and normal subsets. Once these subsets are defined, preprocessing techniques are applied to normalize the data, address class imbalance, and prepare it for subsequent modeling.

The core idea is that anomalies are few and different, thus they are more susceptible to isolation and typically have shorter path lengths in the constructed isolation trees (iTrees).

The path length $h(x)$ for a given instance x is defined as the number of edges traversed from the root node to the external node where the traversal ends. To evaluate the anomaly level of an instance, the algorithm computes the average path length $E(h(x))$ over a forest of randomly built iTrees.

To normalize $E(h(x))$ across trees of different sizes, it is divided by the average path length $c(n)$ of unsuccessful searches in a Binary Search Tree (BST), where n is the number of training instances used to build each iTree [8, 14, 15]. The value of $c(n)$ is given by:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \quad \text{where } H(i) \approx \ln(i) + \gamma \tag{1}$$

Here, $H(i)$ is the i-th harmonic number and $\gamma \approx 0.5772$ is the Euler–Mascheroni constant [8, 14, 15]. Using this, the anomaly score $s(x, n)$ of an instance x is defined as:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}} \tag{2}$$

This score lies in the interval $(0, 1]$: - If $E(h(x)) \to 0$, then $s(x, n) \to 1$, strongly indicating an anomaly. - If $E(h(x)) \to c(n)$, then $s(x, n) \to 0.5$, suggesting normality. - If $E(h(x)) \to n - 1$, then $s(x, n) \to 0$, indicating high normality.

This formulation ensures that anomaly scores are consistent across datasets of different sizes and tree structures.

The Isolation Forest model is implemented using the `scikit-learn` library [3]. The following hyperparameters are configured for the model:

- `n_estimators`: The number of trees in the ensemble. It is set to $n = 100$, meaning 100 trees are used. - `max_samples`: The number of samples per tree, which is set to $\min(256, \text{number of samples})$. This ensures that each tree is built using a manageable subset of the data. - `contamination`: The proportion of anomalies in the dataset, which is set to 0.1. This implies that the algorithm expects 10% of the data to be anomalous. - `random_state`: A seed for reproducibility, which is set to `random_state` = 42.

Dataset Division and Preprocessing

After training the Isolation Forest model on the complete dataset X, the model is used to classify the samples into two subsets based on the anomaly scores. Let $X_{\text{anomalous}}$ represent the subset containing anomalous data points, and X_{normal} represent the subset of normal data points. These subsets are defined as follows:

$$X_{\text{anomalous}} = \{x_i \mid s(x_i) \geq \tau\}, \quad X_{\text{normal}} = \{x_i \mid s(x_i) < \tau\} \tag{3}$$

where $s(x_i)$ is the anomaly score for each data point x_i, and τ is the anomaly threshold. If $s(x_i) \geq \tau$, the data point is classified as anomalous; otherwise, it is classified as normal.

The preprocessing step involves several key operations to ensure that the dataset is properly prepared for modeling.

Feature Normalization To ensure consistency in the scale of the features, normalization is applied to the dataset. This step ensures that all features lie within a similar range, preventing any one feature from dominating the model due to its scale. Min-Max scaling is used, which transforms each feature x_{ij} to a scaled value x'_{ij} according to the formula:

$$x'_{ij} = \frac{x_{ij} - \min(X_j)}{\max(X_j) - \min(X_j)} \tag{4}$$

where x_{ij} is the value of feature j for sample i, and $\min(X_j)$ and $\max(X_j)$ are the minimum and maximum values of feature j across all samples [22].

The process is followed by dimensionality reduction with incremental PCA to reduce the dimensionality of the dataset while retaining as much of the variance as possible, incremental principal component analysis (PCA) is applied. Incremental PCA is particularly useful for large datasets, as it can process the data in mini-batches. The technique projects the data into a lower-dimensional space where the maximum variance of the data is preserved [19]. This step helps reduce the complexity of the model and ensures that important features are retained while less relevant ones are discarded.

This work also proposes the use of class balancing with stratified subsampling and SMOTE to address class imbalance, stratified subsampling is applied first. This technique ensures that both normal and anomalous classes are well-represented in both training and test datasets by preserving their relative proportions. After the initial class balance is achieved, the Synthetic Minority Oversampling Technique (SMOTE) is applied to generate synthetic data points for the minority class [4, 17]. SMOTE works by creating synthetic samples as linear combinations of existing minority class samples and their nearest neighbors. The formula for generating a synthetic sample x_{syn} is:

$$x_{\text{syn}} = x_i + \lambda(x_k - x_i), \quad \lambda \sim \text{Uniform}(0, 1) \tag{5}$$

where x_i is a minority class sample, x_k is one of its k nearest neighbors, and λ is a random factor drawn from a uniform distribution between 0 and 1. This approach helps balance the class distribution and improves the performance of the machine learning model [4, 17].

Impact of Isolation Forest and SMOTE Figure **??** shows the distribution of classes in the dataset after the application of the Isolation Forest algorithm. As seen in the graph, the data is separated into normal and anomalous classes based on the anomaly scores, but with a significant imbalance between them.

In the first path, SMOTE effectively addresses the high initial class imbalance caused by the Isolation Forest separation, balancing the data by generating synthetic samples for the anomalous class. This process significantly reduces the underrepresentation of the anomalous class and ensures a more balanced dataset for subsequent modeling and evaluation.

Classification Models

Once both datasets were balanced, the classification stage involved using two distinct models: Random Forest (RF) and a hybrid Convolutional Neural Network with Long Short-Term Memory (CNN-LSTM). These models were applied separately to each dataset to evaluate their performance.

The RF model was trained on the balanced datasets, ensuring that the influence of underrepresented classes was mitigated. The model's performance was evaluated using key metrics, including precision, recall, and F1-score.

In the CNN-LSTM approach, the features were reshaped to accommodate the input structure required by the model, transforming the data into a three-dimensional format. The architecture of the CNN-LSTM included a ConvLSTM2D layer followed by dense layers, enabling the model to learn both spatial and temporal dependencies in the data. The datasets were divided into training and validation sets to monitor the model's performance and detect potential overfitting during the training process. It is important to note that certain labels were removed due to their underrepresentation in the datasets, a result of the balancing process performed earlier. This adjustment was necessary to avoid biases and ensure the models could effectively generalize across the remaining classes.

2.2 Second Path

The second processing path follows the exact same steps as the first, but with an alteration in the order of operations. The primary difference lies in the order in which preprocessing and anomaly detection are applied. Additionally, class imbalance is addressed using manual sampling, followed by the application of the Synthetic Minority Over-sampling Technique (SMOTE). This helps to balance the dataset, ensuring better model performance on the minority class.

3 Results

3.1 First Path Results

The application of Isolation Forest demonstrated efficient anomaly detection, as summarized in Table 2, which includes training times and performance metrics

Table 2: Model Training Times, Evaluation Metrics, and Isolation Forest Performance

Step	**Time (seconds)**
Isolation Forest Fitting	88.39
Anomaly Prediction	70.54
Data Splitting	0.03
Training Random Forest (Normal)	14.70
Training Random Forest (Anomalous)	8.26
Evaluation Random Forest (Normal)	0.71
Evaluation Random Forest (Anomalous)	0.35
Training CNN-LSTM (Normal)	164.00
Training CNN-LSTM (Anomalous)	94.00
CNN-LSTM Total Execution Time for Normal Dataset	**234.54**
CNN-LSTM Total Execution Time for Anomalous Dataset	**197.38**

for all models. Following preprocessing steps, the data was prepared for training, ensuring compatibility for the subsequent models. of classes for anomalous and normal data after the application of Isolation Forest. The figure **??** shows the class distribution after Isolation Forest application on clean data, and the bottom graph shows the distribution of anomalies after Isolation Forest separation. After initially applying the Isolation Forest algorithm, the class distribution became increasingly imbalanced, particularly affecting the minority classes. This imbalance resulted in the complete disappearance of several underrepresented categories.

The issue was further intensified during the stratified subsampling stage, where classes such as Bot, Web Attack, and Brute Force were entirely eliminated due to their low representation, as illustrated in Figure **??**.This further highlights the challenges in maintaining a balanced dataset when applying Isolation Forest, which disproportionately affects the minority classes. Furthermore, the model training and evaluation times for both the normal and anomalous data pipelines—after applying the Isolation Forest—are summarized in Table 2. This includes the time required for fitting the Isolation Forest, predicting anomalies, and training both Random Forest and CNN-LSTM models on separated data.

Random Forest Results

The performance of the Random Forest model on both normal and anomalous datasets is summarized in Table 3. For the normal dataset, the model demonstrated consistently high precision, recall, and F1-scores across all classes, with an overall accuracy of 99%. Similarly, for the anomalous dataset, the model achieved near-perfect classification, with precision, recall, and F1-scores reaching 1.00 for most classes and an overall accuracy of 100%. These results highlight the

Table 3: Comparison of Random Forest Classification Performance on Normal and Anomalous Data

Class	**Normal Data**				**Anomalous Data**			
	Precision	Recall	F1-Score	Support	Precision	Recall	F1-Score	Support
BENIGN	0.99	0.97	0.98	2676	0.99	1.00	1.00	2672
Bot	0.99	1.00	0.99	2700	–	–	–	–
Brute Force	1.00	1.00	1.00	2677	–	–	–	–
DDoS	1.00	1.00	1.00	2619	1.00	1.00	1.00	2745
DoS	0.99	0.99	0.99	2760	1.00	0.99	1.00	2673
Port Scan	0.99	1.00	0.99	2689	1.00	1.00	1.00	2710
Web Attack	0.99	1.00	1.00	2779	–	–	–	–
Accuracy		0.99				1.00		
Macro Avg	0.99	0.99	0.99	18900	1.00	1.00	1.00	10800
Weighted Avg	0.99	0.99	0.99	18900	1.00	1.00	1.00	10800

robustness of the Random Forest classifier in both clean and anomaly-injected scenarios.

CNN-LSTM Classification Performance

Table 4 compares the performance of the CNN-LSTM model on both normal and anomalous datasets. On the normal dataset, the model achieved an overall accuracy of 98%, with slightly lower recall for the BENIGN class. On the anomalous dataset, the model reached a higher overall accuracy of 99%, showing consistent improvements across most classes. These results suggest that the CNN-LSTM model performs slightly better when trained on data with anomalies explicitly identified by the Isolation Forest algorithm.

3.2 Second Path Results

The second approach involves preprocessing the dataset before applying the Isolation Forest for anomaly detection. This method streamlines the data preparation process and improves model compatibility. In this case, if the dataset is preprocessed before applying Isolation Forest, no classes are eliminated, as can be seen in Figure **??**. In this case, preprocessing before the anomaly detection step ensures that all classes remain intact, and the class distribution is more balanced, with no significant loss of data. Table 5 summarizes the execution times for key steps in the pipeline. The preprocessing step was efficient, and the classification models were trained and evaluated within a reasonable timeframe.

Table 4: Comparison of CNN-LSTM Classification Performance on Normal and Anomalous Data

Class	**Normal Data**				**Anomalous Data**			
	Precision	Recall	F1-Score	Support	Precision	Recall	F1-Score	Support
BENIGN	0.99	0.90	0.94	1786	0.99	0.98	0.99	1801
Bot	0.98	1.00	0.99	1846	–	–	–	–
Brute Force	1.00	0.97	0.98	1773	–	–	–	–
DDoS	0.99	1.00	0.99	1810	0.99	1.00	0.99	1818
DoS	0.96	1.00	0.98	1781	0.98	0.99	0.98	1785
Port Scan	0.99	1.00	0.99	1810	0.99	0.98	0.98	1796
Web Attack	0.96	0.99	0.98	1794	–	–	–	–
Accuracy			0.98				0.99	
Macro Avg	0.98	0.98	0.98	12600	0.99	0.99	0.99	7200
Weighted Avg	0.98	0.98	0.98	12600	0.99	0.99	0.99	7200

Random Forest Results

Table 6 summarizes the performance of the Random Forest classifier under both normal and anomalous data conditions in the second path. The model achieved outstanding accuracy (99%) in both scenarios. In the normal dataset, precision, recall, and F1-score were consistently high across all classes. In the anomalous dataset, although the overall performance remained strong, minor variations were observed, particularly in classes with very low support, such as Brute Force. This indicates that while the classifier generalizes well to anomalous data, rare classes may benefit from additional oversampling or feature enhancement techniques to improve recall.

CNN-LSTM Results

Table 7 presents a comparison of the CNN-LSTM model's classification performance under normal and anomalous data conditions in the second path. The model demonstrated consistent accuracy of 98% in both scenarios. In the normal dataset, performance was slightly lower for minority classes such as Brute Force due to limited support. However, in the anomalous dataset—after preprocessing and rebalancing—performance improved significantly across all classes, including previously underrepresented ones. These results highlight the CNN-LSTM model's robustness and its ability to benefit from data enhancement techniques applied in this path.

Compared to previous works, our approach provides a balance between complexity and efficiency. For instance, Khan et al. [12] proposed a Conv-LSTM-based IDS with an accuracy of 97.29% on the ISCX-UNB dataset, which is

Table 5: Execution Times for Second Path

Step	**Time (seconds)**
Isolation Forest Fitting	1.97
Anomaly Prediction	1.47
Data Separation	0.01
Saving Datasets	1.71
Data Splitting	0.01
Training Normal Model	15.45
Training Anomalous Model	1.14
Evaluation Normal Model	0.59
Evaluation Anomalous Model	0.05
CNN-LSTM Total Execution Time for Normal Dataset	**2.44**
CNN-LSTM Total Execution Time for Anomalous Dataset	**1.32**

slightly lower than our method's performance. Their model leverages temporal and spatial traffic patterns but comes with higher computational costs. Similarly, Rosay et al. [20] introduced an MLP-based intrusion detection model with an impressive accuracy of 99.95% on CIC-IDS2017. However, MLP models are often limited in handling complex sequential patterns compared to CNN-LSTM architectures.

Nawaz et al. [18] addressed class imbalance using SMOTE with LSTM, achieving 99.33% accuracy. While their model effectively handles under-represented classes like Brute Force and DDoS attacks, the focal loss function increases computational demands. In contrast, our approach, incorporating Isolation Forest for preprocessing, reduces computational load while maintaining competitive accuracy. Furthermore, we observe that classes like Heartbleeding, which are severely under-represented, are not suitable for reliable model application. Based on our analysis, we conclude that it is better to exclude classes with so few samples, as they do not provide sufficient data for effective model training and could potentially introduce noise. Table 8 summarizes the differences between our work and related studies, focusing on accuracy, model complexity, and computational efficiency.

4 Conclusions and Future Work

This study presents a hybrid IDS evaluation approach combining Isolation Forest as an additional preprocessing step for Random Forest and CNN-LSTM models. The methodology effectively handles high-dimensional data and achieves high accuracy in detecting anomalies within the CIC-IDS2017 dataset. While our Random Forest model provides superior performance in terms of speed, CNN-

Table 6: Comparison of Random Forest Classification Performance on Normal and Anomalous Data (Second Path)

Class	**Normal Data**				**Anomalous Data**			
	Precision	Recall	F1-Score	Support	Precision	Recall	F1-Score	Support
BENIGN	0.99	0.98	0.99	2449	0.98	0.97	0.98	298
Bot	0.99	1.00	1.00	2588	0.97	1.00	0.98	148
Brute Force	1.00	1.00	1.00	2698	1.00	0.67	0.80	3
DDoS	1.00	1.00	1.00	2098	1.00	1.00	1.00	566
DoS	0.99	0.99	0.99	1974	1.00	0.99	0.99	727
Port Scan	0.99	1.00	1.00	2670	1.00	1.00	1.00	47
Web Attack	1.00	1.00	1.00	2537	1.00	1.00	1.00	98
Accuracy			0.99				0.99	
Macro Avg	0.99	0.99	0.99	17014	0.99	0.95	0.96	1887
Weighted Avg	0.99	0.99	0.99	17014	0.99	0.99	0.99	1887

LSTM improves detection of complex, sequential attack patterns. Compared to related works, our model strikes a balance between simplicity and performance, maintaining a high detection rate with lower computational costs.

Despite the strong performance metrics reported across studies, including ours, a notable limitation lies in the lack of evaluation against adversarial attacks. High accuracy and recall do not guarantee robustness when models are exposed to intentionally crafted inputs designed to evade detection. This gap highlights a potential vulnerability in current ML-based NIDS approaches, suggesting that future research should complement traditional evaluations with adversarial robustness assessments to ensure real-world reliability.

Future work should focus on the development of an automated end-to-end pipeline capable of handling real-time network traffic for intrusion detection. This pipeline should integrate preprocessing, anomaly detection, and classification in a streamlined workflow suitable for deployment in production environments. Given its consistent performance, the Random Forest classifier is a strong candidate for real-world implementation. A critical challenge in practical deployments is the presence of adversarial crafted attacks inputs designed to evade detection by manipulating feature values in subtle ways. As machine learning becomes increasingly embedded in cybersecurity infrastructure, defending against such attacks is essential to maintain system integrity. Isolation Forest can limit the impact of adversarial perturbations by reducing exposure to manipulated traffic, thus enhancing the overall resilience of the intrusion detection system. Future research should also explore adversarial training strategies, combining synthetic adversarial examples with real traffic data to further strengthen the model's robustness under adversarial conditions.

Table 7: Comparison of CNN-LSTM Classification Performance on Normal and Anomalous Data (Second Path)

Class	**Normal Data**				**Anomalous Data**			
	Precision	Recall	F1-Score	Support	Precision	Recall	F1-Score	Support
BENIGN	0.98	0.90	0.94	207	0.99	0.92	0.95	1613
Bot	0.94	1.00	0.97	88	0.97	1.00	0.99	1671
Brute Force	1.00	0.67	0.80	3	0.99	0.97	0.98	1786
DDoS	1.00	0.99	1.00	359	0.98	1.00	0.99	1450
DoS	0.97	1.00	0.99	569	0.96	1.00	0.98	1219
Port Scan	1.00	1.00	1.00	20	0.99	1.00	0.99	1779
Web Attack	1.00	1.00	1.00	14	0.97	0.99	0.98	1822
Accuracy			0.98				0.98	
Macro Avg	0.98	0.94	0.96	1260	0.98	0.98	0.98	11340
Weighted Avg	0.98	0.98	0.98	1260	0.98	0.98	0.98	11340

Table 8: Comparison with Related Works

Reference	**Algorithm**	**Dataset**	**Accuracy (%)**	**Recall (%)**
[12]	Conv-LSTM	ISCX-UNB	97.29	97.29
[20]	MLP	CIC-IDS2017	99.95	99.35
[18]	LSTM + SMOTE	CIC-IDS2017	99.33	98.22
Our Work	IForest + RF/CNN-LSTM	CIC-IDS2017	99.0–100.0	98–99

References

1. Alotaibi, A., Rassam, M.A.: Adversarial machine learning attacks against intrusion detection systems: A survey on strategies and defense. Future Internet **15**(2), 62 (2023)
2. Apruzzese, G., Andreolini, M., Ferretti, L., Marchetti, M., Colajanni, M.: Modeling realistic adversarial attacks against network intrusion detection systems. Digital Threats: Research and Practice (DTRAP) **3**(3), 1–19 (2022)
3. Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., Niculae, V., Prettenhofer, P., Gramfort, A., Grobler, J., Layton, R., VanderPlas, J., Joly, A., Holt, B., Varoquaux, G.: API design for machine learning software: experiences from the scikit-learn project. In: ECML PKDD Workshop: Languages for Data Mining and Machine Learning. pp. 108–122 (2013)
4. Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: Smote: synthetic minority over-sampling technique. Journal of artificial intelligence research **16**, 321–357 (2002)

5. Chen, Z., Zhou, L., Yu, W.: Adasyn- random forest based intrusion detection model. In: Proceedings of the 2021 4th International Conference on Signal Processing and Machine Learning. pp. 152–159 (2021)
6. Durumeric, Z., Li, F., Kasten, J., Amann, J., Beekman, J., Payer, M., Weaver, N., Adrian, D., Paxson, V., Bailey, M., et al.: The matter of heartbleed. In: Proceedings of the 2014 conference on internet measurement conference. pp. 475–488 (2014)
7. Farnaaz, N., Jabbar, M.: Random forest modeling for network intrusion detection system. Procedia Computer Science **89**, 213–217 (2016)
8. Hariri, S., Kind, M.C., Brunner, R.J.: Extended isolation forest. IEEE transactions on knowledge and data engineering **33**(4), 1479–1489 (2019)
9. Heidari, A., Jabraeil Jamali, M.A.: Internet of things intrusion detection systems: a comprehensive review and future directions. Cluster Computing **26**(6), 3753–3780 (2023)
10. Jose, J., Jose, D.V.: Deep learning algorithms for intrusion detection systems in internet of things using cic-ids 2017 dataset. International Journal of Electrical and Computer Engineering (IJECE) **13**(1), 1134–1141 (2023)
11. Kachoukh, A., Albalawi, N.S., Alshammari, S.A.: Advanced convolutional neural network and long short-term memory model for real-time spam detection in internet of things devices. J. Electrical Systems **20**(10s), 6114–6135 (2024)
12. Khan, M.A., Karim, M.R., Kim, Y.: A scalable and hybrid intrusion detection system based on the convolutional-lstm network. Symmetry **11**(4), 583 (2019)
13. Khan, M.A., Kim, Y.: Deep learning-based hybrid intelligent intrusion detection system. Computers, Materials & Continua **68**(1) (2021)
14. Liu, F.T., Ting, K.M., Zhou, Z.H.: Isolation forest. In: 2008 eighth ieee international conference on data mining. pp. 413–422. IEEE (2008)
15. Liu, F.T., Ting, K.M., Zhou, Z.H.: Isolation-based anomaly detection. ACM Transactions on Knowledge Discovery from Data (TKDD) **6**(1), 1–39 (2012)
16. Louati, F., Barika Ktata, F., Amous, I.: An intelligent security system using enhanced anomaly-based detection scheme. The Computer Journal p. bxae008 (2024)
17. Mansourifar, H., Shi, W.: Deep synthetic minority over-sampling technique. arXiv preprint arXiv:2003.09788 (2020)
18. Nawaz, M.W., Munawar, R., Mehmood, A., Rahman, M.M.U., Abbasi, Q.H.: Multi-class network intrusion detection with class imbalance via lstm & smote. arXiv preprint arXiv:2310.01850 (2023)
19. Rehman, A., Khan, A., Ali, M.A., Khan, M.U., Khan, S.U., Ali, L.: Performance analysis of pca, sparse pca, kernel pca and incremental pca algorithms for heart failure prediction. In: 2020 International Conference on Electrical, Communication, and Computer Engineering (ICECCE). pp. 1–5. IEEE (2020)
20. Rosay, A., Carlier, F., Leroux, P.: Mlp4nids: An efficient mlp-based network intrusion detection for cicids2017 dataset. In: Machine Learning for Networking: Second IFIP TC 6 International Conference, MLN 2019, Paris, France, December 3–5, 2019, Revised Selected Papers 2. pp. 240–254. Springer (2020)
21. Rosay, A., Cheval, E., Carlier, F., Leroux, P.: Network intrusion detection: A comprehensive analysis of cic-ids2017. In: 8th international conference on information systems security and privacy. pp. 25–36. SCITEPRESS-Science and Technology Publications (2022)
22. Shantal, M., Othman, Z., Bakar, A.A.: A novel approach for data feature weighting using correlation coefficients and min–max normalization. Symmetry **15**(12), 2185 (2023)

23. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., et al.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. ICISSp **1**, 108–116 (2018)
24. Verkerken, M., D'hooge, L., Wauters, T., Volckaert, B., De Turck, F.: Unsupervised machine learning techniques for network intrusion detection on modern data. In: 2020 4th Cyber Security in Networking Conference (CSNet). pp. 1–8. IEEE (2020)
25. Vrskova, R., Hudec, R., Kamencay, P., Sykora, P.: A new approach for abnormal human activities recognition based on convlstm architecture. Sensors **22**(8), 2946 (2022)
26. Wang, S.: Isolation forest anomaly detection algorithm based on multi-level sub-subspace partition. International Journal of Computer Science and Information Technology (2024). https://doi.org/10.62051/ijcsit.v4n2.20
27. Ye, W., Yang, L., Duan, L., Li, C., Ni, W.: Detecting unknown threats in smart contracts with domain adaptation pp. 104–114 (2024). https://doi.org/10.1109/sse62657.2024.00027
28. Yu, X., Huang, Y., Zhang, Y., Song, M., Jia, Z.: Network intrusion traffic detection based on feature extraction. Computers, Materials & Continua **78**(1) (2024)